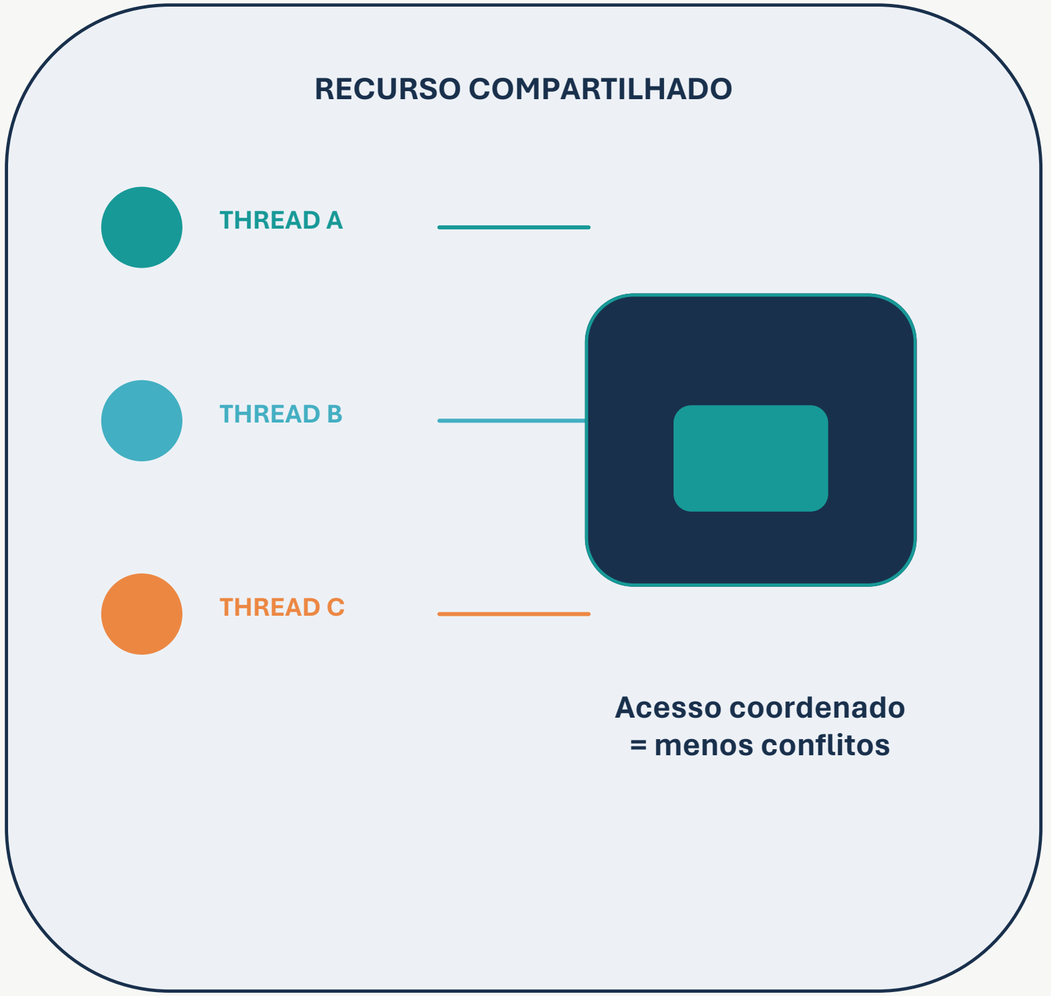
Sincronização de Threads

Como coordenar execuções concorrentes, proteger dados compartilhados e evitar impasses.

CONCORRÊNCIA • COORDENAÇÃO • RECURSOS



Aluna: Emília Ribeiro Santos

O que é Sincronização de Threads

Sincronização de threads significa coordenar a execução de diferentes threads para que elas trabalhem de maneira organizada, principalmente quando precisam compartilhar dados ou recursos.

Imaginem, por exemplo, duas operações tentando modificar a mesma informação em um sistema ao mesmo tempo. Se não houver um controle adequado, uma operação pode interferir na outra e produzir um resultado incorreto.

Por isso, a sincronização é importante para proteger os dados compartilhados, organizar o acesso aos recursos e evitar problemas durante a execução dos programas.

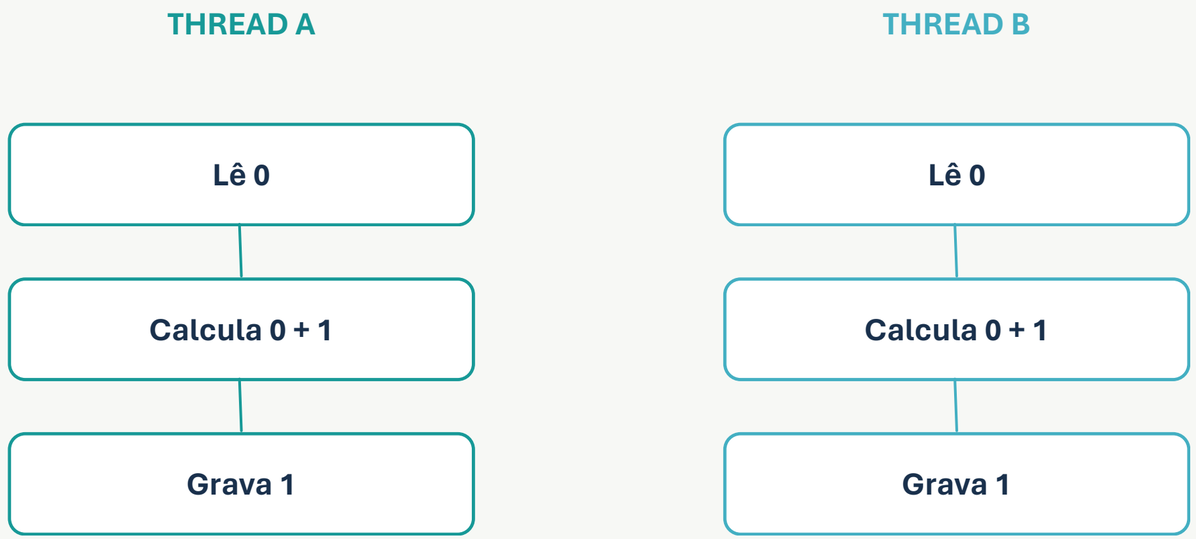
Condição de corrida: quando a ordem muda o resultado

O resultado depende da ordem em que operações concorrentes são intercaladas.

Exemplo: contador compartilhado começa em 0,
duas threads tentam incrementá-lo.

CONTADOR COMPARTILHADO

valor = 0



Resultado: 1 — embora o esperado fosse 2

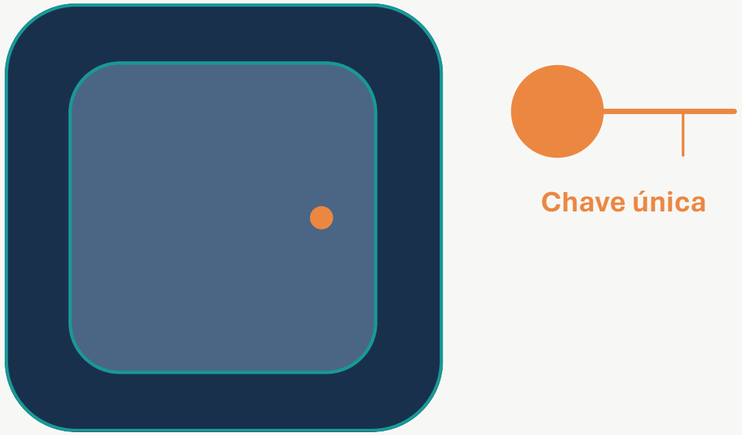
Uma gravação sobrescreve a outra, ocorre uma atualização perdida.

Para evitar isso, precisamos controlar o acesso à parte do código que modifica os dados compartilhados.

Exclusão mútua: uma chave para a seção crítica

Princípio que permite que apenas uma thread por vez acesse o recurso protegido.

Analogia: banheiro com uma única chave. Quem entra usa o espaço e devolve a chave ao sair.



Chave única

SEÇÃO CRÍTICA

É onde ocorre o acesso ao dado ou recurso compartilhado que precisa de proteção.



Exclusão mútua é o princípio, mutex é um mecanismo usado para implementá-la.

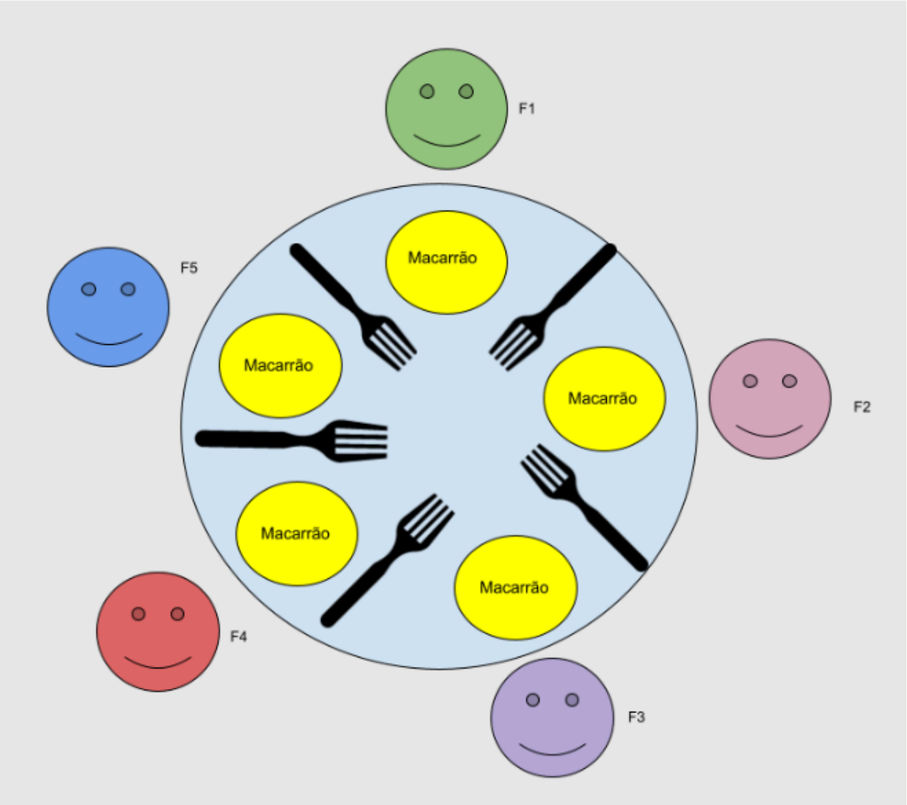
O jantar dos filósofos: o cenário do problema

Cinco filósofos compartilham cinco garfos. Para comer, cada um precisa dos dois garfos que estão ao seu lado.

Regras do modelo:
- Cada filósofo alterna entre pensar e comer.
- Só pode pegar um garfo por vez.
- Um garfo não pode ser usado por dois filósofos ao mesmo tempo.

Esse cenário permite visualizar problemas de sincronização. Dependendo de como os filósofos pegam os garfos, eles podem ficar bloqueados ou algum pode esperar por muito tempo.

Os filósofos representam as threads, e os garfos representam os recursos compartilhados



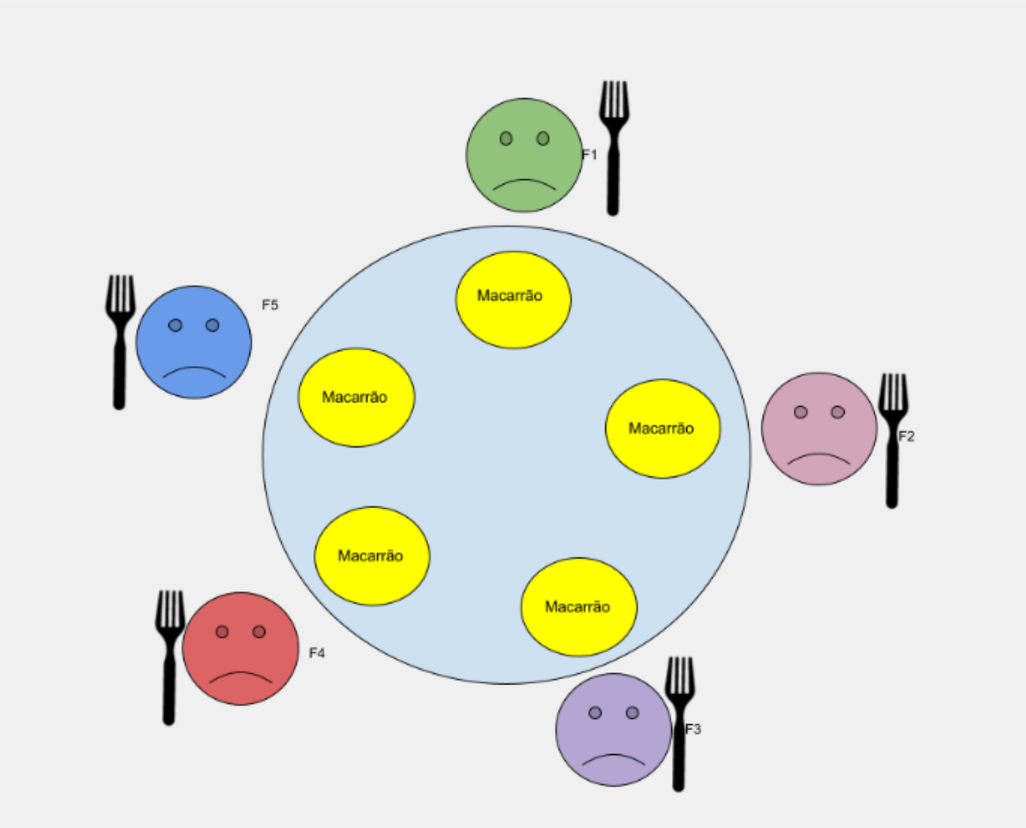
Este cenário ajuda a visualizar deadlock, starvation e sincronização.

Deadlock: todos esperam, ninguém come

Deadlock significa impasse. Acontece quando duas ou mais threads ficam bloqueadas esperando por recursos que estão sendo mantidos umas pelas outras.

Se cada filósofo pegar o garfo à esquerda e esperar pelo da direita, forma-se uma dependência circular.

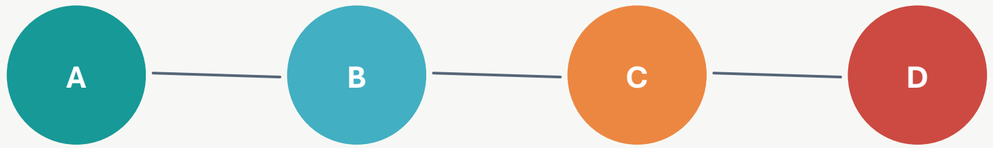
Cada um mantém um recurso e aguarda outro que está com o vizinho. Ninguém libera o que possui. Assim, todos ficam bloqueados.



Prevenção: impor uma ordem ou usar um coordenador.

Starvation: uma thread fica sem oportunidade

O sistema continua funcionando, mas uma thread pode esperar indefinidamente porque outras passam à frente.



THREAD D

continua esperando

A diferença entre os dois, no deadlock, os envolvidos ficam presos em um impasse. Na starvation, o sistema continua funcionando, mas uma thread pode ficar sempre para trás.

Deadlock = impasse
Starvation = espera injusta de uma thread

Mecanismos de Sincronização: Mutex e semáforos

MUTEX

Controla o acesso exclusivo a uma região protegida.



Acquire → executar → release

Exemplo: proteger o saldo bancário.
Ele pode proteger a parte do código responsável por atualizar o saldo, evitando que operações simultâneas interfiram umas nas outras.

SEMÁFORO CONTADOR

Controla quantas unidades de um recurso estão disponíveis.



Wait: adquirir uma unidade

Signal: liberar/sinalizar

Exemplo: três computadores para vários usuários.
O semáforo começa com o valor três. Quando os três computadores estão ocupados, quem chegar depois precisa esperar até que um seja liberado.

Então, a diferença principal é: o mutex controla o acesso exclusivo a uma região protegida, enquanto o semáforo pode controlar a quantidade de recursos disponíveis.

Outros mecanismos de sincronização

EVENTOS

Uma thread sinaliza que algo aconteceu.

Ex.: arquivo terminou de carregar.

VARIÁVEIS DE CONDIÇÃO

Espera até uma condição ser verdadeira.

Ex.: consumidor espera a fila ter itens disponível.

BARREIRAS

Todas as threads precisam chegar a um ponto.

Ex.: Isso pode ser útil quando diferentes partes de uma imagem estão sendo processadas em paralelo.

JOIN

Uma thread aguarda outra terminar.

Ex.: a principal espera o processamento.
Por exemplo, a thread principal pode aguardar a conclusão de outras tarefas antes de reunir os resultados.

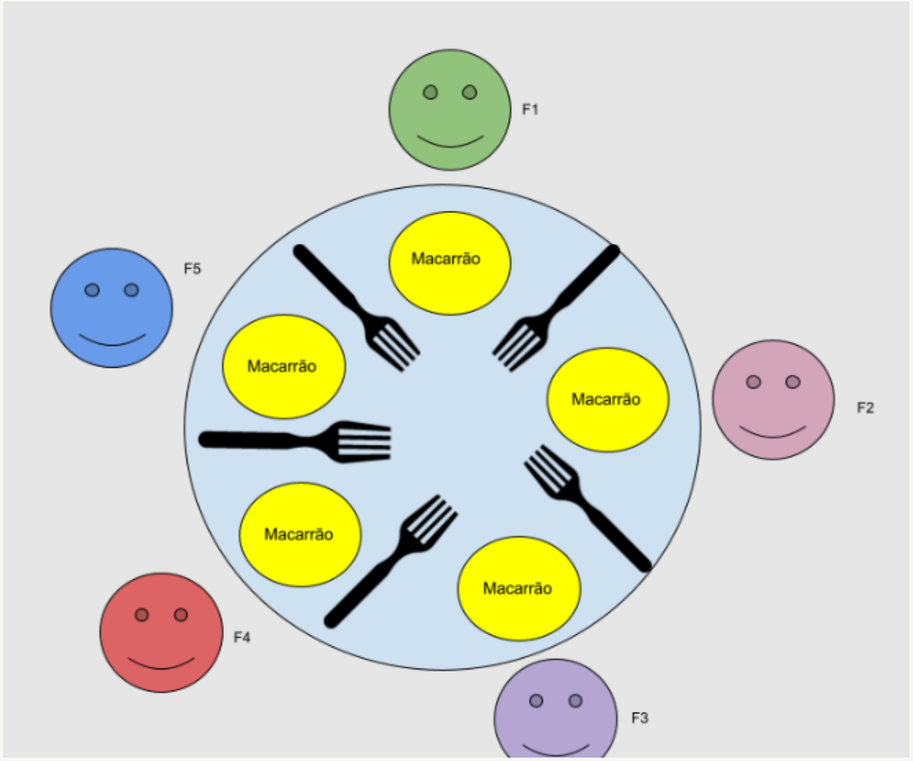
Cada mecanismo serve para uma necessidade diferente: sinalizar acontecimentos, esperar condições, sincronizar etapas ou aguardar o término de uma thread

Como coordenar os filósofos?

O objetivo é organizar o acesso aos garfos para que os filósofos possam progredir.

Estratégias possíveis:

Ordem fixa	Pegar primeiro o garfo de menor e depois o de maior número.
Limitar concorrência	No máximo quatro tentam pegar garfos ao mesmo tempo.
Coordenador	Autoriza a combinação de garfos antes de comer.



Boa sincronização reduz impasses e favorece justiça.

Mapa de decisão: qual mecanismo usar?

A escolha depende do problema que precisamos resolver.



Obrigado!